

EN3150 Assignment 03

Simple convolutional neural network to perform classification.

Sampath K. Perera
September 11, 2026

1 Resource-Constrained CNN for Edge Image Classification

In this assignment, you will create a lightweight image classifier using a convolutional neural network (CNN) designed for embedded or low-resource devices (e.g., microcontrollers, Raspberry Pi, or edge nodes). You have the freedom to use any programming language and toolkit of your choice, but it is recommended to use **Python** along with **TensorFlow/Keras** or **PyTorch**. A simple CNN topology for image classification is shown in the Figure 1.

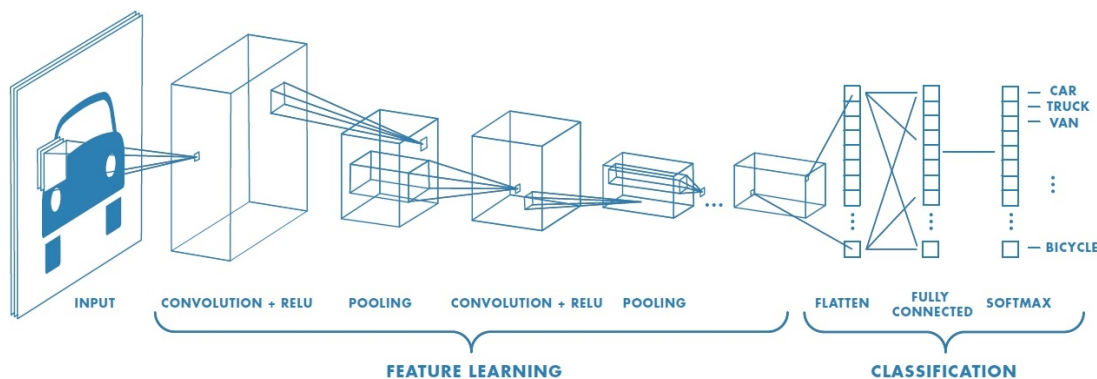


Figure 1: A simple CNN topology for image classification.

1. **Data Preparation:** Choose a dataset from the **UCI Machine Learning Repository** or similar platforms appropriate for low-resolution image classification (**Do not use CIFAR-10 dataset**). To emulate a low-resource sensor environment, downscale your images to a maximum resolution of 64×64 pixels. Split the dataset into training (70%), validation (15%), and testing (15%) sets. Update information about your dataset in [this sheet](#).
2. **Custom Architecture Design** **[20 marks]**
Design and detail two variations of a custom CNN:
 - **Model A (Standard CNN):** Interleave standard 2D convolutional layers and max-pooling layers.

- **Model B (Lightweight CNN):** Replace the standard convolutions with Depth-wise Separable Convolutions to drastically reduce multiply-accumulate (MAC) operations. Your total trainable parameter count for Model B must not exceed 100,000.

Explicitly detail the network parameters (kernel sizes, filters, fully connected units) and calculate the total trainable parameters for both models. Provide hardware-aware justifications for your chosen activation functions (e.g., avoiding computationally expensive exponentials).

3. **Optimizer Selection & Tuning** **[15 marks]**

State which optimizer and learning rate you selected for training and why. Compare the performance of your chosen optimizer against (a) standard Stochastic Gradient Descent (SGD), and (b) SGD with Momentum. Clearly explain the impact of the momentum parameter on your model's convergence and performance.

4. **Custom Model Training & Evaluation** **[25 marks]**

Train both models for at least 20 epochs and plot the training/validation loss curves. Evaluate both models on the test set and record the accuracy, confusion matrix, precision, and recall.

Comparison: Create a table comparing Model A and Model B on total parameter count, estimated model size on disk (KB), training time per epoch, and test accuracy. Discuss the trade-offs observed when moving from standard to depthwise separable convolutions.

2 Compare with Lightweight State-of-the-Art Networks

"In practice, deploying heavy state-of-the-art models like VGG or standard ResNet on edge devices is often impossible due to strict memory constraints. Instead, the industry relies on specialized, lightweight architectures utilizing parameter-efficient operations to achieve high accuracy with a fraction of the computational cost."

5. **SOTA Fine-Tuning & Evaluation** **[20 marks]**

Choose two pre-trained **lightweight** architectures designed for mobile/edge computing (e.g., **MobileNetV1/V2**, **SqueezeNet**, **EfficientNet-B0**, or **ShuffleNet**). Fine-tune and train both models using the same data splits as your custom networks. Evaluate them on the test set and record their performance metrics, specifically indicate the total parameter count and model size (MB).

6. **Final Comparison & Trade-off Discussion** **[20 marks]**

Compare your custom resource-constrained CNN (Model B) with the fine-tuned lightweight SOTA models. Discuss the trade-offs, advantages, and limitations of building a custom sub-100k parameter model from scratch versus using an industry-standard lightweight pre-trained model. Focus heavily on the balance between Accuracy, Memory footprint, and Computational cost.

3 Additional Resources

1. [MIT: Convolutional Neural Networks](#)
2. [MIT: Introduction to deep learning](#)
3. [Pytorch training a classifier](#)
4. [Pytorch build a model](#)
5. [Deep learning with Pytorch: A 60 minute blitz](#)
6. [keras image classification from scratch](#)
7. [keras image classification via fine-tuning with EfficientNet](#)
8. [Conv Nets: A Modular Perspective](#)
9. [Understanding Convolutions](#)
10. [pytorch resnet](#)
11. [pytorch googlenet](#)
12. [Pytorch models and pre-trained weights](#)
13. [Transfer learning for computer vision tutorial](#)
14. [Stanford: Transfer learning](#)

4 GitHub Profile

- You **must** include the link to your GitHub (or some other SVN) profile, so that I can see that you have worked on this assignment over a reasonable duration. Therefore, make commits regularly. However, I will use only the pdf for grading to save time.

5 Submission

- Upload a report and your code separately to the Moodle. Use file name format as "Yourgroupno_A03_EN3150". Ensure that your code is complete, well-documented (commented), and ready to run. Include the index numbers and the names of the group members within the report. Only one submission is required from a group.
- The interpretation of results and the discussion are important in the report.

- Plagiarism will be checked and in cases of plagiarism, an extra penalty of 50% will be applied. In case of copying from each other, both parties involved will receive a grade of zero for the assignment. Academic integrity is of utmost importance, and any form of plagiarism¹ or cheating will not be tolerated.
- An extra penalty of 20% is applied for late submission.

References

- [1] Kevin P Murphy, *Probabilistic machine learning: an introduction*, MIT press, 2022.
- [2] Kunihiro Fukushima, “Cognitron: A self-organizing multilayered neural network,” *Biological cybernetics*, vol. 20, no. 3-4, pp. 121–136, 1975.
- [3] David H Hubel and Torsten N Wiesel, “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex,” *The Journal of physiology*, vol. 160, no. 1, pp. 106, 1962.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

¹<https://en.wikipedia.org/wiki/Plagiarism>